PROJECT TEAM-G 7 COOKER SHOP WEBSITE SYSTEM

Asmaa Daher

Manager





Misk Mashahreh

Programmer

Masa Awad

Technical Architect





Anwar Radwan

Requirement Engineer

Abdel Rahman Sabbagh

Secretary



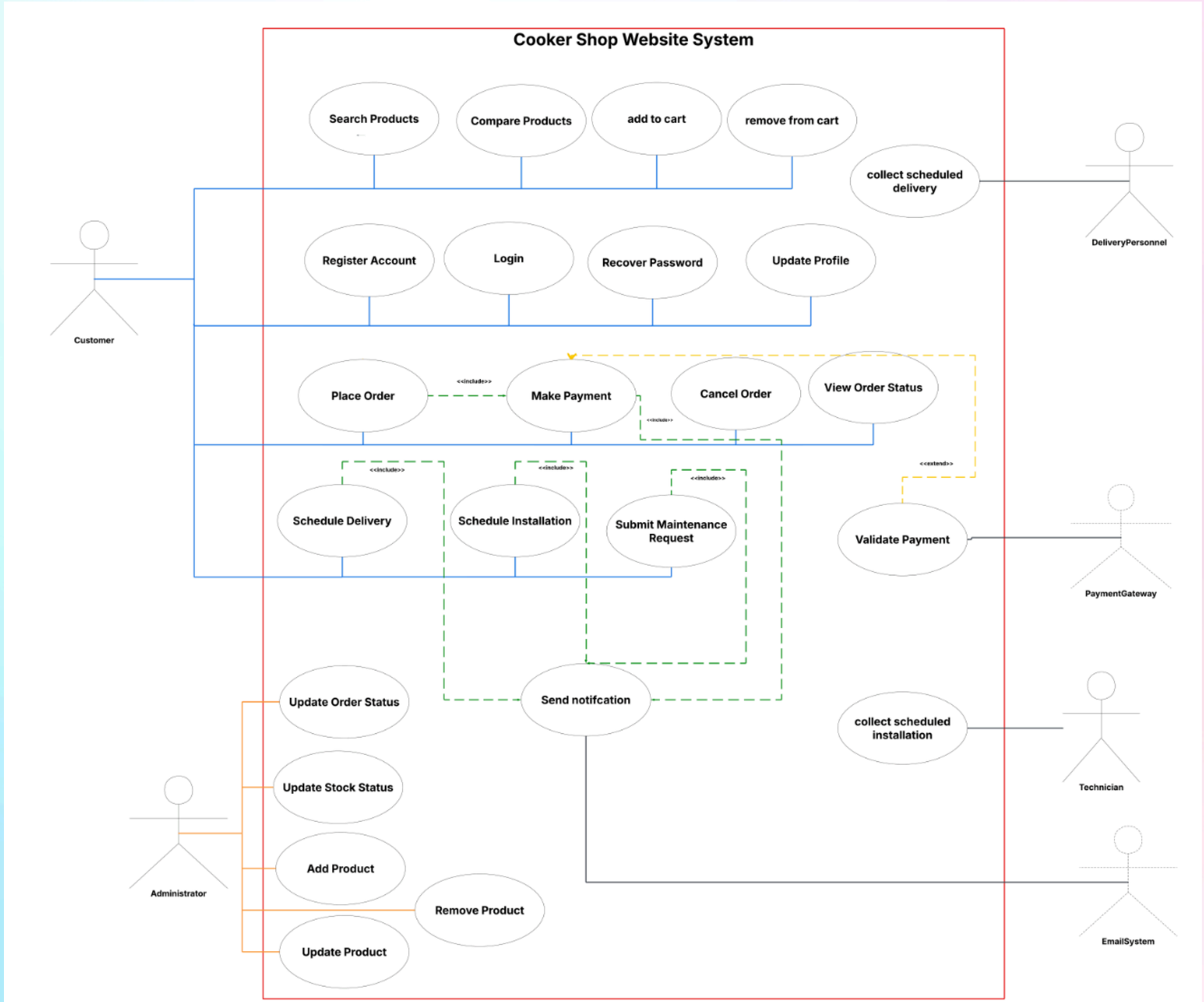
Business description



The Cooker Shop is a business that sells cookers and related kitchen appliances to individual and business customers through an online website. Customers can browse products, compare specifications, place orders, make payments, and schedule delivery and installation services. The business also provides maintenance services for purchased products. The system helps manage product catalogs, customer orders, payments, deliveries, and maintenance requests efficiently while ensuring a reliable and convenient customer experience.



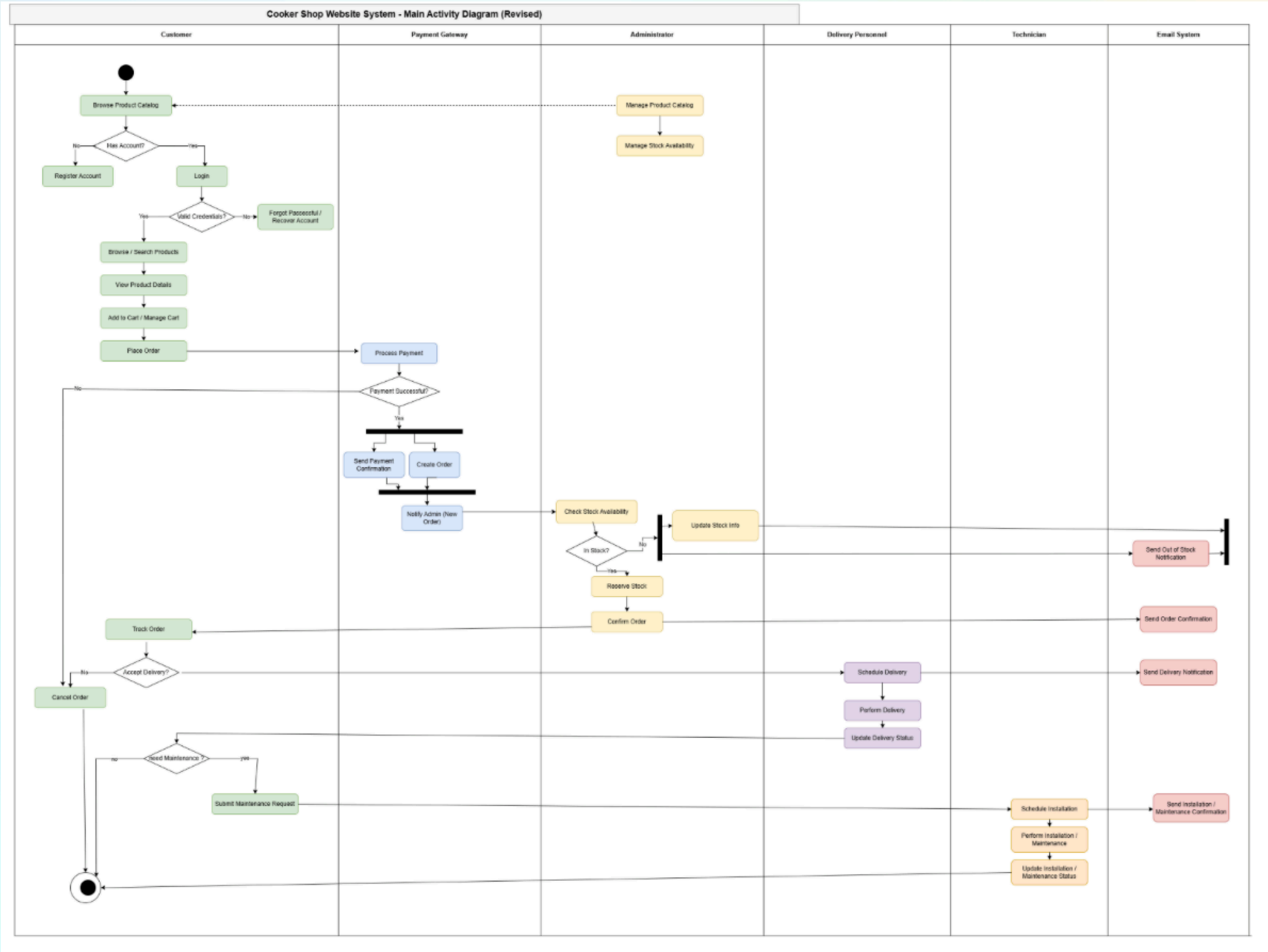
Use Case Diagram



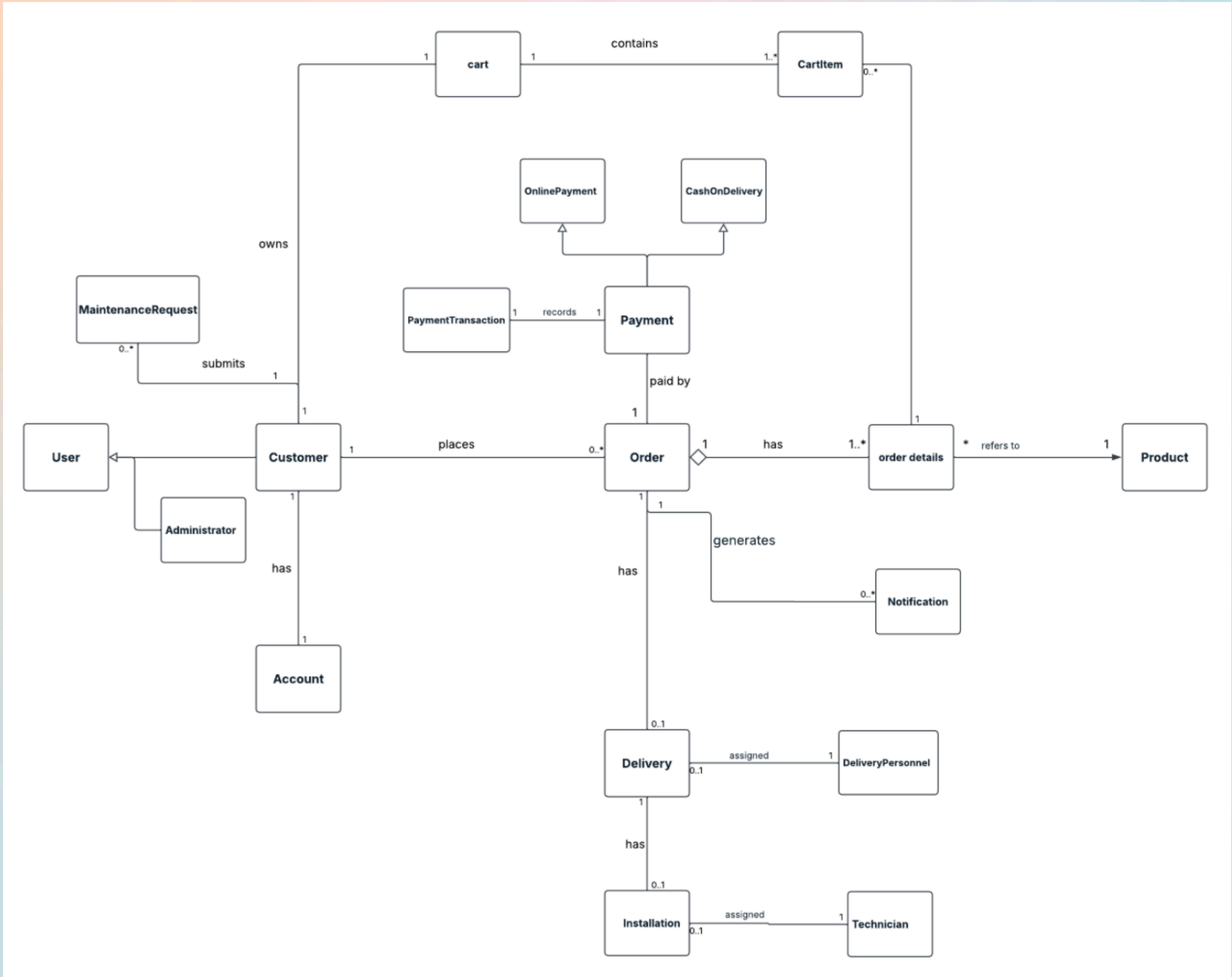
Example of a Use Case Description

Field	Content
System Name	Cooker Shop Website System
Use Case Title	Place Order
Actors	Customer ,PaymentGateway ,EmailSystem
Data	Order details, delivery address, phone number, selected products, quantities, payment details
Stimulus / Trigger	Customer submits an order for processing.
Pre-conditions	.1 Customer is logged in to a registered account . 2 .Shopping cart contains at least one in-stock product . .3 System is online and accessible . 4 .Product inventory service is operational. 5. Payment processing service is available.
Description	This use case describes the process by which a registered customer confirms and submits an order for one or more cooker products. The system validates the order details, processes the payment through the Payment Gateway ,generates a unique order identifier, and dispatches a confirmation email to the customer via the EmailSystem
Workflow / Flow of Events	Normal Flow : 1 .The customer reviews the order summary showing product details, quantities, total cost, and delivery address . .2 The customer clicks "Confirm Order ." 3 .The system validates all required fields including delivery address and phone number . 4 . The system verifies the availability and quantity of each selected product in the inventory. .5 The system sends the order total and payment transaction request to the PaymentGateway for authorization. . 6 .The PaymentGateway confirms successful payment . .7 The system generates a unique order ID and records the order . 8 .The EmailSystem dispatches an order confirmation to the customer's registered email address . 9 .The system displays a success message with the order ID to the customer .
	Error Flow : 3a. If any required field is missing or invalid → the system highlights the missing field and displays an error message. The use case returns to step 1 . 4a. If one or more products are out of stock → the system rejects the order, notifies the customer, and returns them to the cart . 6a. If payment fails → the PaymentGateway returns a failure response → the system notifies the customer and returns to step 1. The order is not recorded . Alternative Flow: 7a. Customer chooses to save the order information for later checkout. 7b. System stores the pending order and allows the customer to complete payment later.
Post-conditions	.1 A new order is recorded in the system with a unique order ID . .2 The order status is set to "Placed ." .3 An order confirmation email has been sent to the customer. 4. Stock availability is updated for all ordered products .
Comments	Customer must be authenticated before placing an order. Out-of-stock products cannot be ordered .

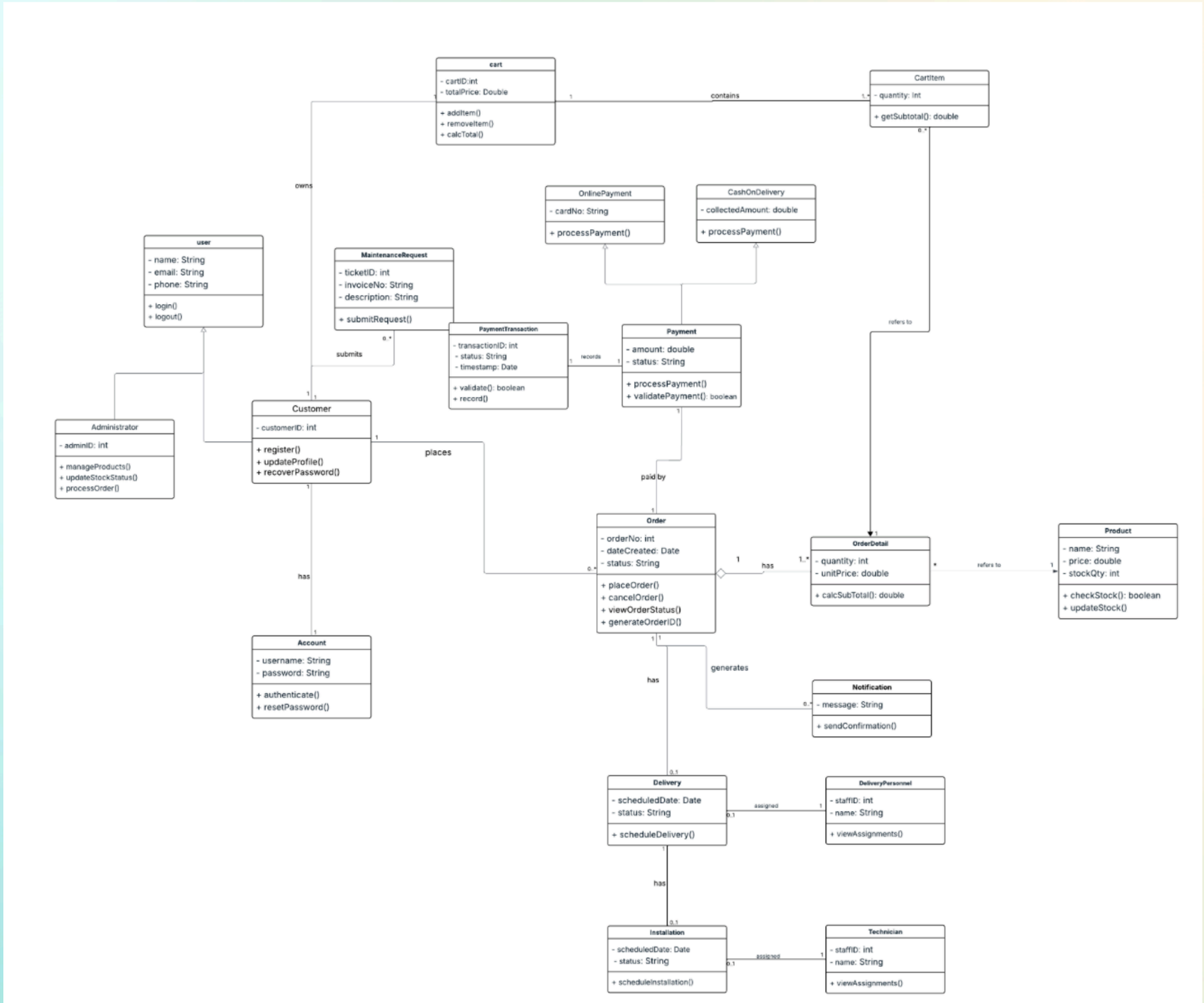
Activity Diagram



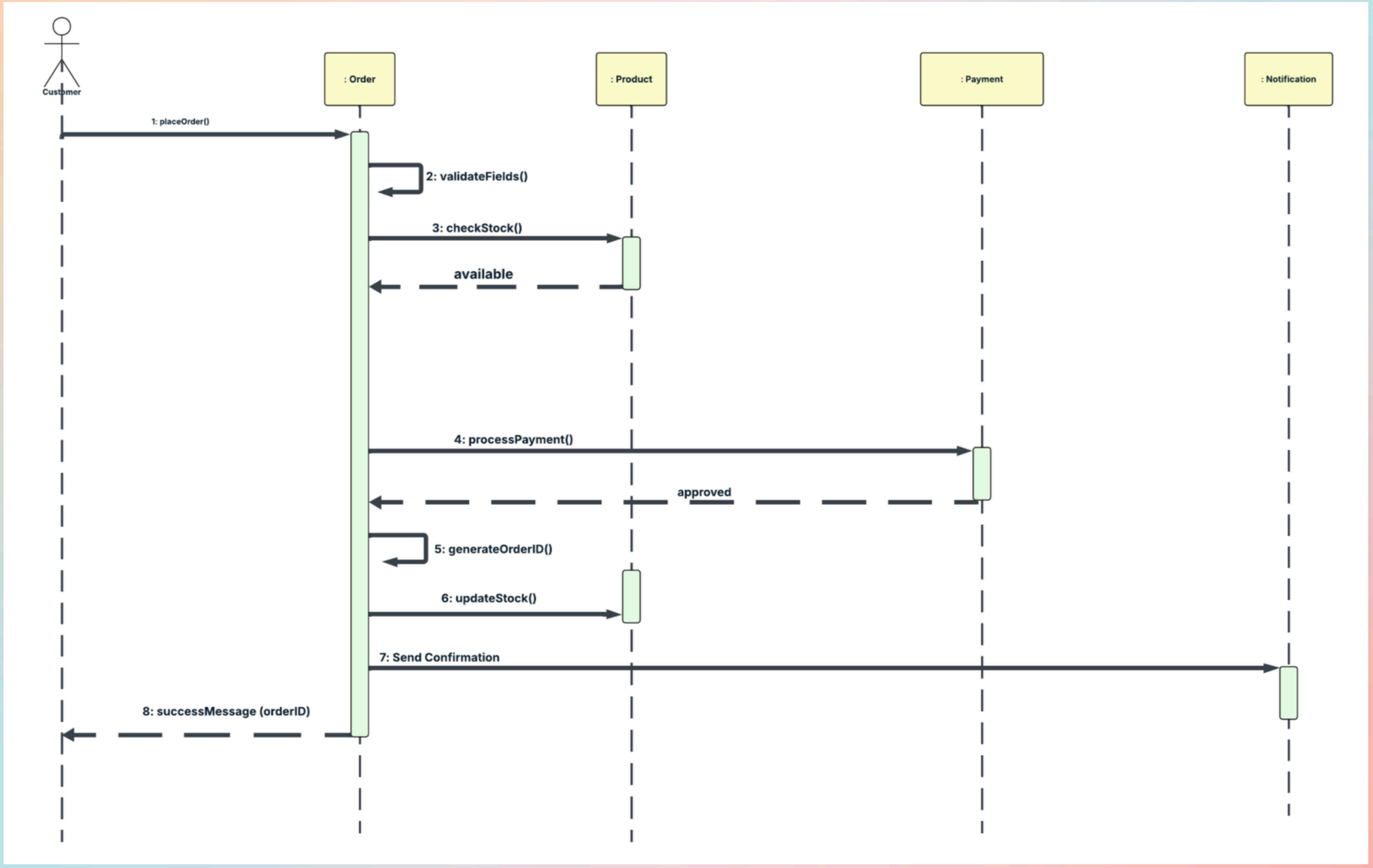
Analysis Class Diagram



Detailed Class Diagram



Sequence Diagram



Design Goals

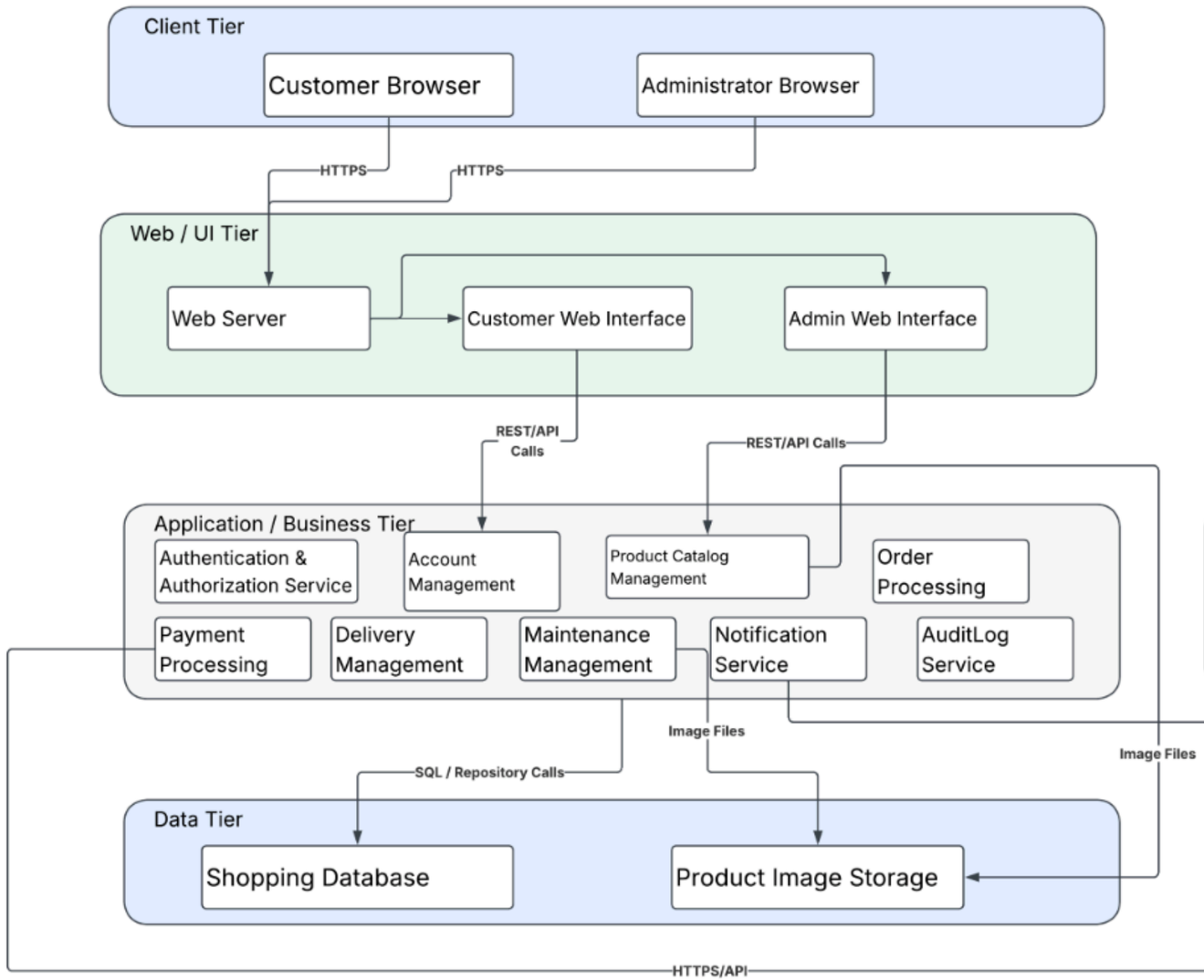
1. Low Coupling
Each tier and component communicates only through well-defined interfaces, and no component accesses another tier's internals directly. For example, the Web/UI Tier never accesses the Data Tier or external services directly — all such calls pass through the Application/Business Tier via REST/API calls. This is addressed in the Architecture Diagram (4-tier layered client-server style) and in the Component Diagram, where components such as Maintenance Management and Order Processing expose narrow interfaces (e.g., submitRequest, placeOrder) rather than sharing internal data structures.

2. High Cohesion
Each service is responsible for one focused business capability. Maintenance Management only handles maintenance requests and warranty verification; it does not also manage orders or payments. This separation is visible in the Component Diagram, where Account Management, Product Catalog Management, Order Processing, Delivery Management, Maintenance Management, Payment Processing, and Notification Service are each isolated as standalone components with a single responsibility, and in the Detailed Class Diagram, where MaintenanceRequest only contains attributes and operations relevant to a maintenance ticket.
3. Security (Non-functional Goal)

Sensitive operations must pass through authentication and authorization, and sensitive data must never be exposed unnecessarily. All external communication uses HTTPS/API, full payment card details are never permanently stored (only handled through the external Payment Gateway), and administrative actions on the product catalog are recorded in an audit log. This is addressed in the Architecture Diagram (Authentication & Authorization Service as a distinct component, HTTPS on all client-server links) and in the Deployment Diagram, where the Payment Gateway and Security Server are reached over separate, isolated channels (HTTPS, TCP/IP) rather than through the database directly.

Architecture Diagram

Chosen architectural style: 4-tier layered client-server architecture

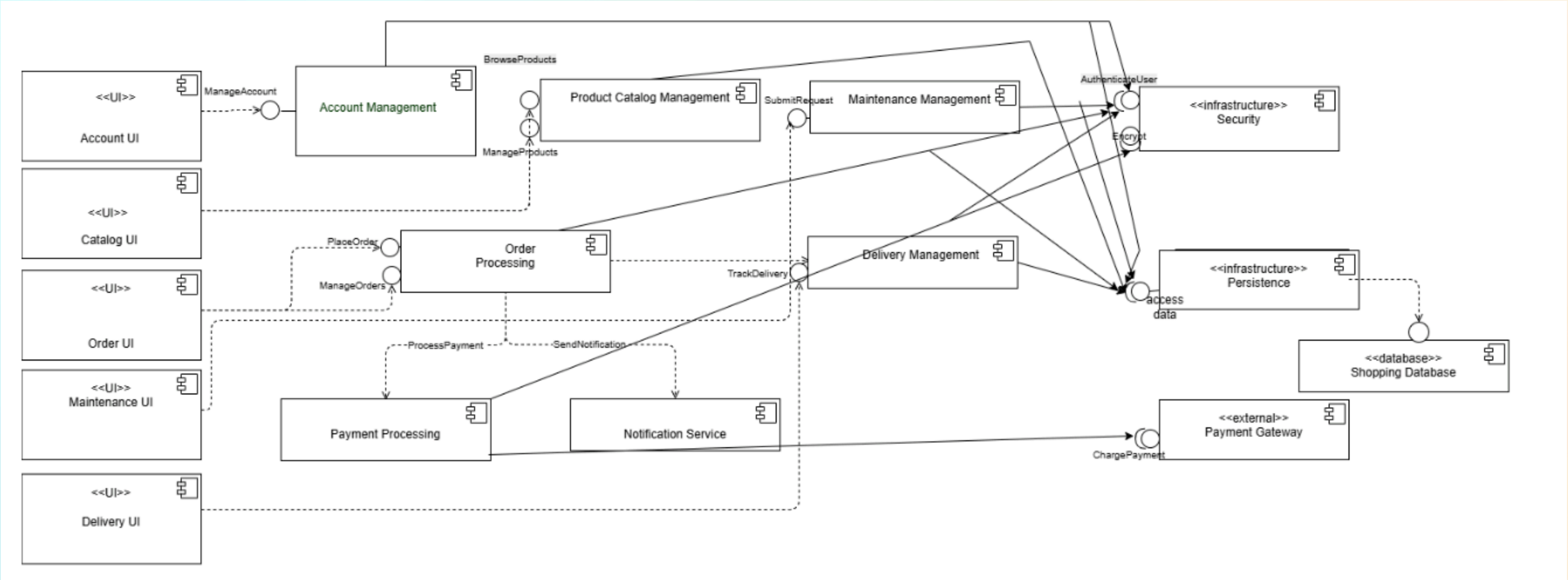


Design Goal 1 — Low Coupling
Each tier communicates through clear interfaces. The UI does not access the database or external services directly.

Design Goal 2 — High Cohesion
Each service has one focused responsibility: product catalog, orders, payment, delivery, maintenance, notification, or audit logging.

Specific Goal — Security
Sensitive operations pass through authentication and authorization. External calls use HTTPS/API. Card details are handled by PaymentGateway. Admin product changes are recorded in AuditLog.

Component Diagram



DEPLOYMENT DIAGRAM

